LECTURE 7

Convolutional Neural Networks

Reference Text

Filters, feature maps, pooling and the architectures that taught machines to see — from pixels to SAR target recognition

AK DHAMIJA, Researcher & Instructor, AI & Machine Learning

dhamija.ak@gmail.com • 9811453163

Unit 4: Simulation Strategies in Warfare — AI & Machine Learning in Warfare

This reference text expands every slide of the lecture into full explanations, derivations, worked examples and figures. Read it alongside the slides; work the "check your understanding" boxes before looking at the answers.

Contents

1 How to Use This Text **3**

2 Images as Grids of Numbers **3**

2.1 Shapes . . . 3

2.2 Why not a plain fully-connected network? . . . 3

3 The Convolution Operation **3**

3.1 The worked example: set-up . . . 4

3.2 Step 1 — three positions in detail . . . 4

3.3 Step 2 — the full feature maps . . . 4

4 Activation and Pooling **5**

4.1 Step 3 — ReLU . . . 5

4.2 Step 4 — max pooling . . . 5

5 Layer Mechanics **5**

5.1 Output size, padding and stride . . . 6

5.2 Channels and filters . . . 6

5.3 Receptive fields . . . 6

5.4 The feature hierarchy . . . 7

6 A Complete CNN: SAR Target Classifier **7**

6.1 Architecture . . . 7

6.2 Output and loss . . . 7

7 Training a CNN **8**

7.1 Backpropagation through a convolution . . . 8

7.2 Practical training . . . 8

8 Landmark Architectures **9**

9 Transfer Learning **9**

10 Defence and Wargaming Applications **10**

11 Failure Modes **10**

11.1 Adversarial examples . . . 10

11.2 Shortcut learning . . . 10

11.3 Distribution shift and over-confidence . . . 10

11.4 Checking what the network looks at . . . 11

12 CNNs and Transformers Compared **11**

13 Common Misconceptions **11**

14 Summary of Equations **12**

15 Check Your Understanding **12**

1 How to Use This Text

This text accompanies the Lecture 7 slides. It assumes Lecture 6: a neuron computes a weighted sum and a non-linear activation, hidden layers learn feature maps, and backpropagation computes every gradient by the chain rule. A convolutional neural network (CNN) uses exactly these ingredients; what is new is the *shape* of its layers, designed for data laid out on a grid. Work through the worked example of Sections 3–4 with a pencil: every number can be checked by hand, and the same four steps (convolve, activate, pool, repeat) underlie every CNN in service.

2 Images as Grids of Numbers

2.1 Shapes

A grey-scale image is a matrix of pixel intensities; a colour image is a three-dimensional array of size height × width × channels. A 224×224 RGB photograph holds $224 \times 224 \times 3 = 150{,}528$ numbers. SAR and thermal (IR) imagery is usually single-channel; multispectral satellite imagery may have 4–13 channels. A wargame map can also be treated as an image whose channels are layers of information: own units, enemy units, terrain, visibility.

2.2 Why not a plain fully-connected network?

1. **Too many weights.** Connecting a $224 \times 224 \times 3$ image to 1,000 hidden units needs $150{,}528 \times 1{,}000 \approx 150$ million weights for the first layer alone — more parameters than there are training images in most defence datasets, an invitation to overfitting.
2. **Neighbourhoods are thrown away.** Flattening the image into a list forgets which pixels are adjacent, yet edges, corners and textures are all *local* patterns.
3. **No tolerance to position.** A vehicle in the top-left corner and the same vehicle in the centre activate entirely different weights, so the network must learn "vehicle" separately at every position.

The CNN answers each problem with one idea: **local connectivity** (each unit looks at a small patch), **weight sharing** (the same filter is used at every position), and a **hierarchy** of layers (patches grow into objects). The first layer of a typical CNN on the same RGB image — 64 filters of size $3 \times 3 \times 3$ — has $(3 \cdot 3 \cdot 3 + 1) \times 64 = 1{,}792$ parameters, about $150{,}528{,}000/1{,}792 \approx 84{,}000$ times fewer, and detects its patterns anywhere in the image. The design was inspired by Hubel and Wiesel's studies of the visual cortex (1959–62), which found neurons that respond to edges of particular orientation in small regions of the visual field.

3 The Convolution Operation

Definition — 2-D convolution as used in CNNs

For an input x and a $k \times k$ kernel K with bias b, the output at position (i, j) is

$$y_{i,j} = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} K_{m,n}\, x_{i+m,\, j+n} + b.$$

The kernel is placed on a $k \times k$ patch, multiplied element by element, summed, and written to the output; it then slides one step and repeats. The array of outputs is a **feature map**.

Each output is the dot product of the kernel with a patch: a *similarity score* that is large when the patch looks like the kernel. An imagery analyst sweeping a stencil across a photograph and noting how well each spot matches is doing the same thing. Strictly, the formula is *cross-correlation*; mathematical convolution flips the kernel first. Because CNN kernels are learned, the flip makes no difference, and every deep-learning library calls the operation "convolution".

3.1 The worked example: set-up

The input X is a 6×6 image of the corner of a bright object (say a vehicle return) on a dark background: rows 0–1 and columns 0–1 are 0, and the 4×4 block in rows 2–5, columns 2–5 is 9.

0	0	0	0	0	0
0	0	0	0	0	0
0	0	9	9	9	9
0	0	9	9	9	9
0	0	9	9	9	9
0	0	9	9	9	9

input X (6×6); outlined: the patch for output $(2, 0)$

Figure 1: the input image.

Two kernels (classical Prewitt edge filters) are used, with stride 1, no padding and bias 0:

$$K_1 = \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix} \text{ (left-edge detector)}, \qquad K_2 = \begin{pmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix} \text{ (top-edge detector)}.$$

K_1 is positive where brightness increases from left to right; K_2 where it increases from top to bottom. A trained CNN learns kernels like these in its first layer; nobody programs them.

3.2 Step 1 — three positions in detail

Worked example — individual outputs of K_1

Position (0,0): the patch is rows 0–2, columns 0–2; only its bottom-right pixel is bright (9) and it meets a +1: $y_{0,0} = 9$.
Position (2,0): rows 2–4, columns 0–2; the right column is bright in all three rows: $y_{2,0} = 9 + 9 + 9 = 27$, the maximum response — a full vertical edge.
Position (2,2): rows 2–4, columns 2–4, all bright: $y_{2,2} = (-9 - 9 - 9) + 0 + (9 + 9 + 9) = 0$.

Key point

A uniform region gives zero: the negative and positive columns of an edge kernel cancel. An edge detector responds only where brightness *changes*.

3.3 Step 2 — the full feature maps

A 3×3 kernel fits in $6 - 3 + 1 = 4$ positions in each direction, so each map is 4×4:

$$X * K_1 = \begin{pmatrix} 9 & 9 & 0 & 0 \\ 18 & 18 & 0 & 0 \\ 27 & 27 & 0 & 0 \\ 27 & 27 & 0 & 0 \end{pmatrix}, \qquad X * K_2 = \begin{pmatrix} 9 & 18 & 27 & 27 \\ 9 & 18 & 27 & 27 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

K_1 fires down the *left side* of the object (columns 0–1); K_2 fires along its *top side* (rows 0–1). The values ramp up $9 \to 18 \to 27$ as more of the edge falls inside the window. Each kernel produces one feature map; a layer with 32 kernels produces 32 maps.

4 Activation and Pooling

4.1 Step 3 — ReLU

After convolution each value passes through $\mathrm{ReLU}(z) = \max(0, z)$. Our two maps are already non-negative, so ReLU leaves them unchanged. Its effect is seen with the mirrored kernel $-K_1$, a *right*-edge detector: on the same image every response flips sign, giving values $-9, -18, -27$ down the left side, and ReLU sets the whole map to zero. The map now says, correctly, "there is no right-hand edge here". ReLU keeps evidence *for* a pattern and discards evidence against it, so each map answers one clear question.

Without a non-linearity, two stacked convolutions are a single convolution (a linear map composed with a linear map, as in Lecture 6), and a deep CNN would be no more powerful than one layer. ReLU's gradient is exactly 1 for positive inputs, so it does not shrink the gradients of deep networks the way the sigmoid's (≤ 0.25) does.

4.2 Step 4 — max pooling

Definition — max pooling

A $p \times p$ max-pooling layer with stride p divides each feature map into non-overlapping $p \times p$ blocks and keeps the largest value in each. Average pooling keeps the mean. Pooling has **no learned parameters**.

Worked example — 2×2 max pooling of the two maps

K_1 map: top-left block $\max(9, 9, 18, 18) = 18$; top-right $\max(0, 0, 0, 0) = 0$; bottom-left $\max(27, 27, 27, 27) = 27$; bottom-right 0. K_2 map: $\max(9, 18, 9, 18) = 18$, $\max(27, 27, 27, 27) = 27$, and 0, 0.

$$\mathrm{pool}(X * K_1) = \begin{pmatrix} 18 & 0 \\ 27 & 0 \end{pmatrix}, \qquad \mathrm{pool}(X * K_2) = \begin{pmatrix} 18 & 27 \\ 0 & 0 \end{pmatrix}.$$

The pooled maps summarise the scene: a left edge in the lower-left quadrant, a top edge in the upper-right.

Pooling buys three things: four times fewer numbers downstream; **tolerance to small shifts** (moving the edge by one pixel inside a block leaves the block's maximum unchanged); and a coarser, more abstract summary. Modern networks often replace pooling with a stride-2 convolution, which downsamples with learned weights.

5 Layer Mechanics

5.1 Output size, padding and stride

Derivation — the output-size formula

Pad the input with p zeros on each side, so its width becomes $n + 2p$. A kernel of width k starting at offset 0 fits while its start is at most $n + 2p - k$. With stride s the starts are $0, s, 2s, \ldots$, so the number of positions is

$$n_{\text{out}} = \left\lfloor \frac{n_{\text{in}} + 2p - k}{s} \right\rfloor + 1.$$

Case	n_{in}	k	p	s	n_{out}
Worked example	6	3	0	1	$(6-3)/1+1=4$
LeNet-5 conv1	32	5	0	1	$(32-5)/1+1=28$
AlexNet conv1	227	11	0	4	$(227-11)/4+1=55$
"Same" padding	224	3	1	1	$(224+2-3)/1+1=224$
Downsampling	224	3	1	2	$\lfloor 223/2 \rfloor + 1 = 112$

Without padding every layer shrinks the map and edge pixels are under-used; "same" padding $p = (k-1)/2$ (for odd k) keeps the size. Stride 2 halves the map.

5.2 Channels and filters

A filter spans *all* input channels: on an input with C_{in} channels, a 3×3 filter has $3 \times 3 \times C_{\text{in}}$ weights, multiplies a 3×3 patch in every channel and adds everything (plus one bias) into one number. A layer of C_{out} filters produces C_{out} maps.

Derivation — parameters in one convolutional layer

Each filter has $k \cdot k \cdot C_{\text{in}}$ weights and one bias; there are C_{out} filters:

$$\text{params} = (k^2 C_{\text{in}} + 1)\, C_{\text{out}}.$$

Worked example — counting parameters

RGB input, 64 filters of 3×3: $(9 \cdot 3 + 1) \times 64 = 1{,}792$. A second layer of 128 filters on those 64 maps: $(9 \cdot 64 + 1) \times 128 = 73{,}856$. A layer of 256 filters of 3×3 on 128 channels: $(9 \cdot 128 + 1) \times 256 = 295{,}168$.

The count does *not* depend on the image size: a CNN trained on 64×64 chips can be run, filter for filter, over a 512×512 scene.

5.3 Receptive fields

Definition — receptive field

The receptive field of a unit is the region of the input image that can affect its value.

One 3×3 layer sees 3×3 pixels; a second 3×3 layer on top sees 5×5; a third sees 7×7. In general, with $r_0 = 1$ and jump $j_0 = 1$, a layer with kernel k and stride s gives

$$r_\ell = r_{\ell-1} + (k-1)\, j_{\ell-1}, \qquad j_\ell = j_{\ell-1}\, s.$$

Three stacked 3×3 layers see the same 7×7 area as one 7×7 layer with $3 \times 9 = 27$ weights instead of 49 per channel pair, and three non-linearities instead of one. This is the design rule of VGG.

Worked example — receptive field of the SAR classifier of Section 6

Conv1 ($k = 3$, $s = 1$): $r = 3$, $j = 1$. Pool1 ($k = 2$, $s = 2$): $r = 3 + 1 \cdot 1 = 4$, $j = 2$. Conv2: $r = 4 + 2 \cdot 2 = 8$, $j = 2$. Pool2: $r = 8 + 1 \cdot 2 = 10$, $j = 4$. Conv3: $r = 10 + 2 \cdot 4 = 18$, $j = 4$. Pool3: $r = 18 + 1 \cdot 4 = 22$, $j = 8$. Each of the 8×8 final positions sees a 22×22-pixel region of the 64×64 chip; global average pooling then combines them all.

5.4 The feature hierarchy

Each layer combines the previous layer's patterns over a larger area: layer 1 detects edges and blobs; layers 2–3 corners and textures; middle layers parts (wheels, barrels, track skirts); deep layers whole vehicles. Zeiler and Fergus (2013) confirmed this by visualising what trained networks respond to. The analogy is the intelligence process: individual sensor returns, then platoon formations, then a battalion's scheme of manoeuvre — each level built on the one below. This is the "features of features" argument of Lecture 6, made spatial.

6 A Complete CNN: SAR Target Classifier

6.1 Architecture

A 10-class classifier for 64×64 single-channel SAR chips:

Layer	Operation	Output shape	Parameters
Input	SAR chip	$64 \times 64 \times 1$	0
Conv1	32 filters 3×3, same, ReLU	$64 \times 64 \times 32$	$(9 \cdot 1 + 1) \cdot 32 = 320$
Pool1	max 2×2	$32 \times 32 \times 32$	0
Conv2	64 filters 3×3, same, ReLU	$32 \times 32 \times 64$	$(9 \cdot 32 + 1) \cdot 64 = 18{,}496$
Pool2	max 2×2	$16 \times 16 \times 64$	0
Conv3	128 filters 3×3, same, ReLU	$16 \times 16 \times 128$	$(9 \cdot 64 + 1) \cdot 128 = 73{,}856$
Pool3	max 2×2	$8 \times 8 \times 128$	0
GAP	global average pooling	128	0
Dense	10 classes, softmax	10	$128 \cdot 10 + 10 = 1{,}290$
Total			**93,962**

Spatial size halves ($64 \to 32 \to 16 \to 8$) while channels double ($32 \to 64 \to 128$): the network trades *where* for *what*. **Global average pooling** (GAP) averages each 8×8 map to a single number, giving a 128-long vector and avoiding a large flatten-then-dense layer. For comparison, one dense layer from the flattened 64×64 chip to 128 units would need $4{,}096 \times 128 + 128 = 524{,}416$ parameters — over five times this entire CNN.

6.2 Output and loss

The dense layer produces one *logit* per class and the softmax converts logits to probabilities, $P(c) = e^{z_c} / \sum_{c'} e^{z_{c'}}$. Training minimises the categorical cross-entropy $L = -\ln P(\text{true class})$, the multi-class form of the binary cross-entropy of Lecture 6.

Worked example — softmax and loss for one chip (illustrative)

Logits for five classes T-72, BMP-2, BTR-70, 2S1 and ZSU-23-4: $(4.1, 1.8, 0.9, 0.2, -0.4)$. Exponentials: 60.34, 6.05, 2.46, 1.22, 0.67, total 70.74. Probabilities: 0.853, 0.086, 0.035, 0.017, 0.009. If the chip is a T-72, $L = -\ln 0.853 = 0.159$; if it were actually a BMP-2, $L = -\ln 0.086 = 2.46$ — a confident mistake costs far more.

7 Training a CNN

7.1 Backpropagation through a convolution

The chain rule of Lecture 6 applies unchanged; the only new feature is weight sharing. Because one kernel is used at every position, its gradient is the *sum* of the contributions from all positions. If $\delta_{i,j} = \partial L/\partial y_{i,j}$ is the error at output (i, j),

$$\frac{\partial L}{\partial K_{m,n}} = \sum_{i,j} \delta_{i,j}\, x_{i+m,\, j+n}, \qquad \frac{\partial L}{\partial b} = \sum_{i,j} \delta_{i,j},$$

which is itself a convolution of the input with the error map. The error passed back to the input is a convolution of the error map with the (flipped) kernel.

Worked example — a one-dimensional convolution, forward and backward

Input $x = (1, 2, 0, 3)$, kernel $(k_0, k_1) = (1, -1)$, $y_i = k_0 x_i + k_1 x_{i+1}$: $y = (1-2,\ 2-0,\ 0-3) = (-1,\ 2,\ -3)$. Suppose the errors arriving from above are $\delta = (0.5,\ -1,\ 0.2)$.
Kernel gradients: $\partial L/\partial k_0 = \sum_i \delta_i x_i = 0.5(1) - 1(2) + 0.2(0) = -1.5$; $\partial L/\partial k_1 = \sum_i \delta_i x_{i+1} = 0.5(2) - 1(0) + 0.2(3) = 1.6$.
Input gradients: x_0 feeds only y_0: $0.5 \cdot 1 = 0.5$. x_1 feeds y_0 (via k_1) and y_1 (via k_0): $0.5(-1) + (-1)(1) = -1.5$. x_2: $(-1)(-1) + 0.2(1) = 1.2$. x_3: $0.2(-1) = -0.2$.

Max pooling passes the gradient only to the input that won the maximum; the others receive zero. For a block $(3, 7, 1, 5)$ the output is 7, and an incoming error δ goes entirely to the position that held the 7. **ReLU** passes the gradient where its input was positive and blocks it elsewhere.

7.2 Practical training

- **Data augmentation:** rotations, flips, shifts, crops, added speckle or noise produce free extra examples and teach the invariances the architecture lacks (rotation, scale).
- **Batch normalisation:** rescales each channel's activations to zero mean and unit variance over a mini-batch, with a learned scale and shift; it stabilises and speeds training.
- **Dropout:** randomly silences a fraction of units during training so that no unit can rely on another; it reduces overfitting.
- **Optimiser and schedule:** stochastic gradient descent with momentum, or Adam, with a learning rate reduced as training progresses.

8 Landmark Architectures

Network	Year	Size	Contribution
LeNet-5	1998	∼60K	conv–pool–conv–pool–dense for handwritten digits; the template for every CNN since
AlexNet	2012	∼60M	ReLU, dropout, GPU training; ImageNet top-5 error 15.3% against 26.2% for the next-best entry
VGG-16	2014	138M	only 3×3 kernels, stacked deep
GoogLeNet	2014	∼7M	Inception modules: parallel kernel sizes, 1×1 convolutions to cut channels
ResNet	2015	up to 152 layers	residual (skip) connections; ensembles reached 3.57% top-5 error

Derivation — why residual connections help

A residual block outputs $y = F(x) + x$, where F is a few convolutional layers. Then

$$\frac{\partial y}{\partial x} = \frac{\partial F}{\partial x} + I,$$

so the gradient always has a direct path through the identity term, however small $\partial F/\partial x$ becomes. This counters the vanishing gradients of Lecture 6 and made networks of 100+ layers trainable. The same skip connection appears in every transformer block (Lecture 9).

9 Transfer Learning

Labelled military imagery is scarce. **Transfer learning** starts from a backbone trained on a large dataset (such as ImageNet) and adapts it.

- **Feature extraction:** freeze the backbone and train only a new classification head — fast, and works with very little data.
- **Fine-tuning:** also update the later backbone layers with a small learning rate — better when more labelled data exist.

It works because early-layer features (edges, corners, textures) are generic; only the top layers are specific to the original task.

Pitfall — optical pre-training for SAR

Radar imagery looks very different from optical photographs: speckle, shadows, layover, specular returns. The benefit of optical pre-training is smaller for SAR and must be checked on held-out SAR data. Synthetic training data from simulators and CAD models is a common supplement, but simulated and real returns differ — a distribution shift of exactly the kind discussed in Lecture 1.

10 Defence and Wargaming Applications

Defence & wargaming application — automatic target recognition (ATR)

Classifying vehicles in SAR chips is the classic case. On the public MSTAR benchmark (10 vehicle classes) published CNNs report accuracies above 95% under its *standard operating conditions*, in which training and test chips come from similar collection geometry and vehicle configurations. Accuracy falls, sometimes sharply, for new depression angles, configurations, articulation or camouflage. The first question to ask of any reported accuracy is how the test set was built.

- **Object detection** in EO/IR imagery: detectors such as Faster R-CNN and YOLO output a box and a class for every object — aircraft on aprons, ships in harbour, vehicles on roads.
- **Change detection and segmentation:** compare images over time; label every pixel (road, building, crater) to update terrain databases for simulations.
- **Wargaming:** the map is an image; CNN policies read unit positions, terrain and visibility layers as channels, the approach used in game-playing systems such as AlphaGo. The DQN of Lecture 10 can use a CNN in exactly this way.
- **Beyond images:** 1-D CNNs on radar pulse trains, acoustic and RF signals (emitter identification); spectrograms treated as images.

11 Failure Modes

11.1 Adversarial examples

Tiny, carefully chosen pixel changes can flip a prediction with high confidence. Goodfellow et al. (2014) showed a panda image classified as a gibbon after a perturbation invisible to the eye, produced by the *fast gradient sign method*

$$x' = x + \varepsilon \operatorname{sign}\big(\nabla_x L(x, y)\big),$$

which uses backpropagation to the *input* to find the direction that most increases the loss. Physical versions — printed patches, painted patterns — have fooled detectors in field tests. An adversary who knows a recogniser is in use has an incentive to exploit this.

11.2 Shortcut learning

A network may key on the background, sensor artefacts or image borders instead of the target. A frequently retold, probably apocryphal, story describes a "tank detector" that had learned the weather in the photographs; the lesson is real even if the story is not.

11.3 Distribution shift and over-confidence

New sensors, seasons, terrain or decoys can collapse performance with no warning, because the softmax still outputs a confident-looking probability. CNNs are often over-confident on inputs unlike their training data.

11.4 Checking what the network looks at

Saliency methods such as Grad-CAM (Selvaraju et al., 2017) colour the pixels that most influenced a decision. If the "hot" region is the background rather than the vehicle, the model is not trustworthy, whatever its accuracy.

Key point

Mitigations: diverse training data, adversarial training, test sets drawn from new conditions, calibration checks, and a human analyst confirming every high-consequence call.

12 CNNs and Transformers Compared

	CNN	Vision transformer (ViT)
Basic operation	convolution over a fixed local window	self-attention: every patch to every patch
Built-in assumption	nearby pixels matter; the same pattern can appear anywhere	none — relations are learned from data
Data needed	works well on small and medium datasets	usually needs large datasets or pre-training
Long-range context	only through depth	from the first layer
Cost vs image size	linear in the number of pixels	quadratic in the number of patches

Both use dot-product similarity, ReLU-type non-linearities, residual connections, softmax with cross-entropy, and backpropagation. Many current vision systems are hybrids: a CNN front end for local features feeding attention layers for global context. For small, specialised defence datasets, a CNN remains a strong and economical first choice. Lectures 8 and 9 develop attention.

13 Common Misconceptions

Pitfall — five errors to avoid

"Engineers design the filters." All kernel values are learned by backpropagation; hand-made edge filters are a teaching device. **"Pooling is learned."** Max and average pooling have no parameters. **"A 3×3 filter looks at 9 numbers."** It looks at $3\times3\times C_{\text{in}}$ numbers — all input channels. **"CNNs are fully shift-invariant."** Convolution is shift-*equivariant* and pooling adds only approximate invariance; rotations and scale changes need augmentation. **"A high softmax probability means a reliable call."** CNNs are often over-confident, especially on unfamiliar inputs.

14 Summary of Equations

Convolution	$y_{i,j} = \sum_{m,n} K_{m,n}\, x_{i+m,j+n} + b$
Activation, pooling	$\mathrm{ReLU}(z) = \max(0, z)$; max pool keeps the largest value per block
Output size	$n_{\text{out}} = \lfloor (n_{\text{in}} + 2p - k)/s \rfloor + 1$
Parameters	$(k^2 C_{\text{in}} + 1)\, C_{\text{out}}$ per convolutional layer
Receptive field	$r_\ell = r_{\ell-1} + (k-1) j_{\ell-1}, \;\; j_\ell = j_{\ell-1} s$
Output	$P(c) = \mathrm{softmax}(W\,\mathrm{GAP}(\text{features}) + b)_c, \;\; L = -\ln P(\text{true class})$
Kernel gradient	$\partial L / \partial K_{m,n} = \sum_{i,j} \delta_{i,j}\, x_{i+m,j+n}$
Residual block	$y = F(x) + x, \;\; \partial y/\partial x = \partial F/\partial x + I$

15 Check Your Understanding

Check your understanding — 1

In the worked example, what is the output at position $(1, 0)$ for K_1, and why?

Answer. The patch is rows 1–3, columns 0–2. Its right column holds $0, 9, 9$ and meets $+1$ weights; the other columns are 0. So $y_{1,0} = 0 + 9 + 9 = 18$: two of the three rows of the window contain the edge.

Check your understanding — 2

A 128×128 input meets a 5×5 kernel with $p = 2$, $s = 2$. What is the output size?

Answer. $\lfloor (128 + 4 - 5)/2 \rfloor + 1 = \lfloor 127/2 \rfloor + 1 = 63 + 1 = 64$, so 64×64.

Check your understanding — 3

How many parameters has a layer of 256 filters of 3×3 on 128 input channels?

Answer. $(3 \cdot 3 \cdot 128 + 1) \times 256 = 1{,}153 \times 256 = 295{,}168$.

Check your understanding — 4

Why does a uniform bright region give 0 with an edge kernel?

Answer. The kernel's weights sum to zero $(-3 + 0 + 3)$, so on a constant patch of value c the output is $c \times 0 = 0$. Edge kernels measure *differences* in brightness, not brightness itself.

Check your understanding — 5

Name two ways an adversary could defeat a CNN-based target recogniser.

Answer. Any two of: adversarial patches or paint patterns that push the input across a decision boundary; decoys built to match what the network keys on; camouflage, netting or configuration changes that move the target out of the training distribution; operating at viewing geometries absent from training; exploiting shortcuts (backgrounds or contexts the network relies on).

Check your understanding — 6

What is the receptive field of one unit after two 3×3 convolutions with stride 1 followed by a 2×2 max pool with stride 2?

Answer. $r = 3$, then $3 + 2 = 5$, then $5 + 1 \times 1 = 6$: a 6×6 region.